

Briefing Notes

This activity will take place in DB G.30 and will take 3 hours. You should attend the session shown on your [personal timetable](#). You will be working individually or in groups of 2 and the maximum total number of students is 100.

Health and Safety

The approved Risk Assessment for this activity, reference number: RA_5815, is available on the University LabCup system: <https://mmu.uk.labcup.net/>.



The class will start with a Health and Safety briefing. Students who miss this will not be allowed to take part until they are briefed - please make sure you arrive in good time.

Attendance Record

You may be required to sign an attendance sheet in the laboratory. Please note that this is for the unit teaching team's local records only and does not replace the requirement for [attendance registration](#) for participating students.

Aim

The aim of this experiment is to:

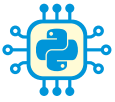
- Introduce the basics of Python programming.

Background Reading

- Thonny, Python IDE for beginners: <https://thonny.org/>
- LearnPython.org interactive Python tutorial: <https://www.learnpython.org/>

About this resource

Document last updated:	September 2026	<i>Note that printed or emailed copies of this document may be out of date. Always check Moodle for the latest version.</i>
Next scheduled review date:	August 2027	
Comments and feedback on this document should be sent to the author(s):		Tim Collins



1 Introduction



Video Introduction to the Module

https://mmutube.mmu.ac.uk/id/1_23wfxlsa

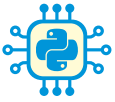
There are many programming languages in use; today, we will be focusing on the language: Python. For programming your robots, we will use the microcontroller version: MicroPython. Python is a high level scripting language that is used for a wide variety of programming tasks including desktop application development, web programming, data science, mathematical modelling, embedded programming, games development and artificial intelligence. It is consistently the number one programming language in demand by employers by a significant margin in the annual IEEE Spectrum survey: <https://spectrum.ieee.org/top-programming-languages-2025>.

There are several Python programming environments available on the University computers that you may come across during your degree such as Visual Studio Code, Idle, Spyder and Eclipse. For this project, we will be using the standalone application: Thonny. This is a capable Integrated Development Environment (IDE), but is easy to pick up as a beginner and can be used for both desktop applications (as we will do today) and for running MicroPython code on connected embedded processors. It is the environment recommended by Raspberry Pi ¹ for developing code for the RP2040 microcontroller which we will be using for your robots.

Thonny is already installed on the laboratory computers. If you want a copy for yourself, it is free to download for Windows, Mac or Linux computers: <https://thonny.org/>

The aim of today's session is to introduce the basics of the programming language by writing some 'Console' applications. These are the simplest applications to develop, having just a text-based 'console' interface.

¹Thonny is installed by default on Raspberry Pi single-board computers running Raspberry Pi OS.



2 Introduction to Python

2.1 Getting Started - 'Hello World'

From the Start menu, find Thonny and run it. There are many 'views' that can be enabled or disabled in Thonny but you will always be able to see the code editing window (labelled '<untitled>' in figure 1) and will need to use the 'Shell' window - if this isn't visible (as in figure 1) select 'Shell' from the 'View' menu and it should appear.



Video Guide: Getting started with Thonny
https://mmutube.mmu.ac.uk/id/1_kwdeliad

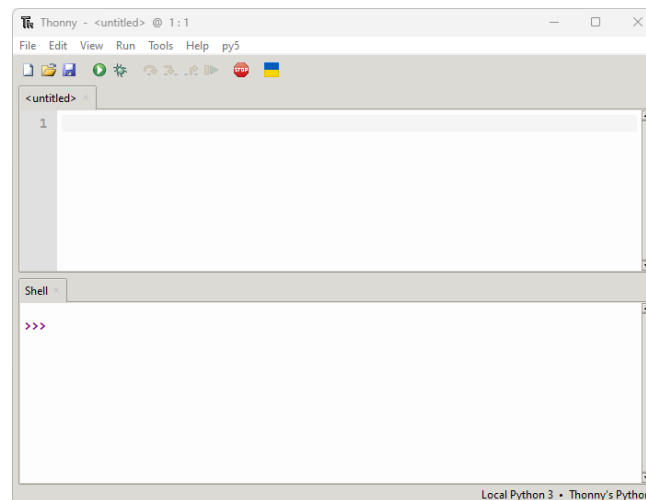


Figure 1: Thonny Python IDE on startup.

The code editing window is where you will write your Python scripts. The Shell window is where you will interact with your scripts when you run them. You can also type and execute Python commands from the Shell window in an interactive fashion (it is often known as a REPL interface: Read, Evaluate, Print, and Loop).

2.1.1 Hello World 1 - Evaluation in the Shell

In the 'Shell' window, type:

```
Shell  
print("Hello World")
```

and press the Enter key. The result should be as shown in figure 2.

`print()` is a built-in Python function that sends whatever is inside the brackets to the console/shell output. The quotation marks around "Hello World" indicate that we want to send a string of characters and the letters between the quotes are the actual message we want to send to the user.

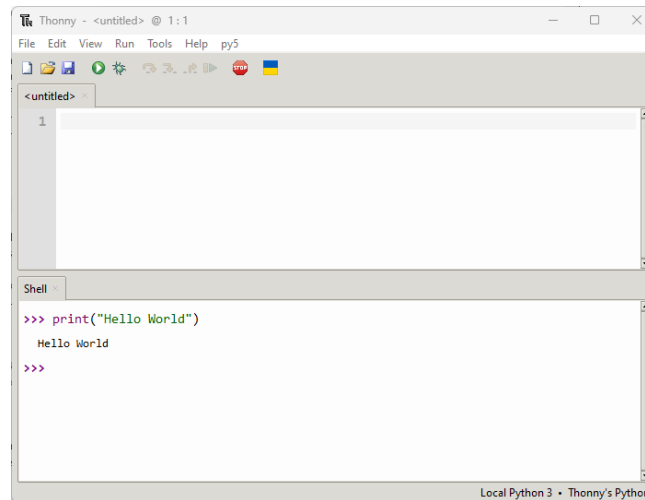
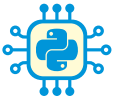


Figure 2: Evaluating `print("Hello World")` from the Shell.

2.1.2 Hello World 2 - Writing a Script

In the code editing window, enter the same line of code as before:

```
1 print("Hello World")
```

and press the Enter key.

This time, nothing will happen. You have written a very short (just one line) script. You now need to *run* the script in order to evaluate the code. To run scripts in Thonny you can either:

1. Select 'Run current script' from the 'Run' menu
2. Click the run icon in the toolbar (the white triangle inside a green circle)
3. Press the F5 key

Whichever method you use, the result should be as in figure 3. 'Hello World' has been written to the Shell window again.

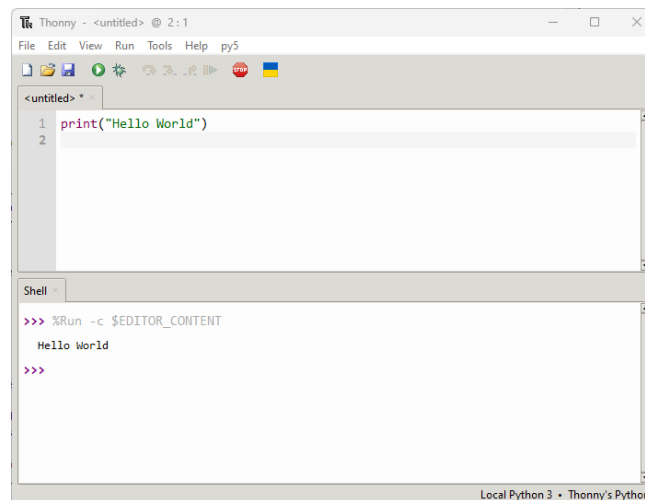
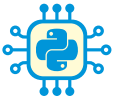


Figure 3: Running a `print("Hello World")` script.

In practice, most of the coding you do will be written as scripts in the code editor in this way although evaluating expressions in the Shell can be very useful when debugging.

2.1.3 Longer Scripts

Any non-trivial script will be longer than a single line of code. A script containing multiple lines is interpreted one line at a time starting from the beginning. Try entering the following three lines into the code editor:

```
1 print("Hello World")
2 print("from")
3 print("Manchester")
```

Run the script and check that the result is what you expected!

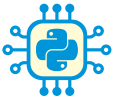
Saving your work regularly is always good practice. Scripts can be saved and reloaded later using the familiar *Save* and *Open* functionality in the file menu. Python scripts are usually given the file extension `.py` to identify them.

Save your latest script in a suitable folder on your OneDrive under an appropriate filename (e.g. `HelloWorld.py`).

Remember:

Shell: Commands entered in the Shell are evaluated immediately.

Script: Commands entered in a script are evaluated one after the other when you run the script. You can run a script multiple times and save it for future use.



2.2 Variables



Video Example: Variables in Python
https://mmutube.mmu.ac.uk/id/1_h48gy6r8

Scripts that perform useful computation will often need to be able to store and retrieve information to and from the computer's memory. This is done using *variables*. Try the following simple example by starting a new script and typing and executing the following code:

```
1 x = 2
2 print("The value of x is")
3 print(x)
```

Running this code should give the result in the shell window:

```
The value of x is
2
```

Looking at this script one line at a time:

```
x = 2
```

This line declares a variable with the label 'x' and allocates some of the computer's memory to store a number associated with the variable. It also assigns the value, 2, to the variable; this value is stored in the memory that has just been allocated.

```
print("The value of x is")
```

Just like the 'Hello World' script, this just outputs the string of characters inside the quotation marks to the shell.

```
print(x)
```

In this line, x is not inside quotation marks so instead of just outputting the character 'x', the print() function outputs the current *value* of the variable, x, i.e. 2.

When using variables, the value returned can be modified by arithmetic operators. For example, to print the value of x plus one:

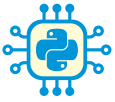
```
1 x = 2
2 print("The value of x plus one is")
3 print(x+1)
```

Note, the third line performs the actual arithmetic. The text in line 2 is purely there for the user's benefit.

Standard arithmetic operators, '+', '-', '*', and '/' can be used to add, subtract, multiply and divide respectively. The operator '**' raises a value to a power and parentheses can also be used, e.g.:

```
1 x = 5
2 print("The value of x squared plus one doubled is")
3 print((x**2 + 1) * 2)
```

This should give the answer: $(x^2 + 1) \times 2 = (5^2 + 1) \times 2 = 26 \times 2 = 52$. Try it and see.



Another useful operator is the modulus operator, '%', which gives the remainder of dividing a number by another. For example:

```
14 % 3
```

is equal to 2 because 14 divided by 3 is 4 remainder 2.

An important point about variables is that their values usually change during the course of a script (hence the name: 'variable'). Try the following example:

```
1 x = 2
2 print(x)
3 x = 13
4 print(x)
5 x = x * 3
6 print(x)
```

The resulting console output should be:

```
2
13
39
```

So, even though the same line of code, `print(x)`, is repeated three times, the resulting console output is different in each case because the value of the variable 'x' changes when its value is (re)assigned by the lines:

```
x = 2
```

(sets the value of x to 2)

```
x = 13
```

(sets the value of x to 13)

```
x = x * 3
```

(sets the value of x to its current value, 13, multiplied by 3 which equals 39)

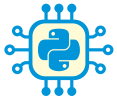
NB. If you want to check the value of a variable, you can just type its name into the Shell and press Enter. In Thonny, you can also view a list of all current variables and their values by selecting 'Variables' from the 'View' menu.

Exercise: Write a script that assigns the value, 4, to a variable, t , and then prints the result of calculating: $3(t^4 - t^2 - t)$
Remember to save your scripts with meaningful filenames so you can find them again later!

2.2.1 Calling Functions

As well as using operators to perform computation on numbers and variables, you can also call *functions*. A function is a block of code that differs from the simple scripts so far in that:

- A function only runs when it is specifically requested, or *called*.
- Many functions can be passed input parameters that change the resulting outcomes.



- Functions can also return values.

You have already used a function: `print()` - a function built into Python that takes a parameter (the contents of the brackets) and displays its value in the Shell.

There are many other built-in functions that we'll use in due course.

Example 1:

Shell

```
x = abs(-3)
print(x)
```

The first line calls the built-in function, `abs()`, with the parameter: `-3`. This function takes a single parameter and returns the absolute value of that parameter. The result is stored in the variable, `x`, and then, on the second line, the value is printed to the Shell. The result should be `'3'`.

Example 2:

Shell

```
m = max(3, 1, 4, 1)
print(m)
```

The function, `max()`, takes two or more parameters as the input and returns the largest of them (i.e. the maximum value). The result of the code above should be `'4'`.

Example 3:

Shell

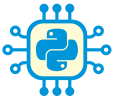
```
help(max)
```

The function, `help()`, takes another function as its parameter and displays a short explanation of the purpose and use of the function. In this case, you should see a few lines explaining how the `max()` function is used.

Exercise: Write a script to find and print the smallest number in the sequence: 4, 7, 3, 8, 2, 4, using the function: `min()`. You may need to use the `help()` function too.

2.2.2 Variable Names

Most scripts will use many variables within them, each with a unique name. Variable names must conform to the following rules:



- A variable name can only contain alpha-numeric characters and underscores (A-Z, a-z, 0-9, and _)
- A variable name must start with a letter or the underscore character (not a number)
- A variable name cannot be any of the Python keywords (e.g. `if`, `while`, `for`, etc).

Upper and lower case characters can be used but note that variable names are case-sensitive. e.g. `mydata` is not the same as `MyData`; they would be stored as two separate variables.

2.3 Data Types



Video Guide: Data types in Python

https://mmutube.mmu.ac.uk/id/1_zi0gqony

The variables used so far have all been storing integer numbers. There are many other types of data that a variable can store. The most commonly used built-in types are:

- **Integers** - 'int' for short. These are whole numbers that can be positive or negative. Fractional values cannot be represented using ints.
- **Floating Point** - 'float' for short. These are numbers that can represent fractional values to a number of significant figures depending on the amount of memory used to represent the number².
- **Boolean** - 'bool' for short. Boolean variables have only two possible values: True or False. They are used in logical operations.

Each of these represents a single value. Other built-in data types that store more than one value include:

- **Complex** - a complex number contains two values: the real part and the imaginary part.
- **String** - 'str' for short. A sequence of text characters than can be of arbitrary length.

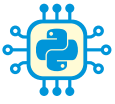
To determine the data type of a variable, there is a function: `type()`. For example:

Shell

```
x = 2
type(x)
y = 3.4
type(y)
a = False
type(a)
z = 1+3j
type(z)
s = 'Hello'
type(s)
```

²You don't need to know how floating point numbers are stored in order to use them in Python. For interest, though, the format is explained here:

<http://mathonline.wikidot.com/storage-of-numbers-in-ieee-single-precision-floating-point-f>



should return:

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'complex'>
<class 'str'>
```

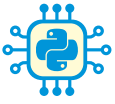
Notes on Data Types:

- If a number is specified without a decimal point, it will be assumed to be an int.
E.g. `x = 3` will create an int variable, x.
- If an int is divided by another int, the result will be a float (even if it is still a whole number)
E.g. `x = 6 / 3` will create a float variable, x with the value, 2.0.
- If you need to do an integer division, the operator `'//'` can be used (any fractional part is lost by rounding down).
E.g. `x = 6 // 3` will create an int variable, x with the value, 2.
- Any number with a decimal point is stored as a float.
E.g. `x = 4.0` will create a float variable, x.
- A float variable can also be declared using scientific notation.
E.g. `x = 3.4e-2` creates a float, x, with the value, $3.4 \times 10^{-2} = 0.034$
- A complex variable can be declared using the syntax: *realPart+imagPartj*
E.g. `x = 3+4j` creates a complex variable with the value $3.0 + 4.0j$
- All the standard operators can be used with complex variables or a mix of complex and float variables.
E.g. `x = (1+2j)**2` creates a complex variable with the value, $(1 + 2j)^2 = -3 + 4j$

2.3.1 Casting Functions

We often want to convert a value from one data type to another (a process known as *casting*). Some commonly used functions to achieve this are:

- `int()` - converts a float or a string to an int.
E.g. `x = int('132')` turns the string of characters into the corresponding int.
- `float()` - converts an int or a string to a float.
E.g. `x = float(4)` turns the int, 4, into a float, 4.0.



- `str()` - converts any other data type to a string of characters.

E.g. `x = str(1+2j)` turns the complex number into a string: "1+2j"

Note that converting to an int will cause any fractional part of a value to be discarded. Also note that some conversions are impossible and will cause an error. E.g. `x = float(2+3j)`

2.4 Console Input

The scripts so far are not interactive; they will produce the same result every time they are executed. We can introduce interactivity using the console input. Try the following code:

```
1 print("Enter the value of x:")
2 x = int(input())
3 print("Enter the value of y:")
4 y = int(input())
5 print("The value of x+y is:")
6 print(x + y)
```

When you execute this script, you will be asked to input values for the two variables 'x' and 'y'. For each one, type an integer number followed by enter. The numbers should appear in the Shell window as you type them. Your script will then calculate their sum.

The line `x = int(input())` contains two functions:

- `input()` tells the interpreter to wait for a sequence of characters to be entered and returns that sequence as a string.
- `int()` converts the inputted string to an integer (miss this out and your input is treated as text rather than a number)

So, `x = int(input())` tells the interpreter to wait for a sequence of characters to be entered, converts them to an integer, and stores it in the variable, `x`.

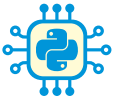
NB. The input function can take a 'prompt string' as a parameter, so the code above could be written more compactly as:

```
1 x = int(input("Enter the value of x: "))
2 y = int(input("Enter the value of y: "))
3 print("The value of x+y is:")
4 print(x + y)
```

Exercise: Write a script that asks the user to enter three numbers: x , y and z . The script should then print the values of:

- i. The sum of x , y and z
- ii. The largest of the three values
- iii. The smallest of the three values

Test to make sure it works correctly.



2.5 Formatted Text



Video Guide: Console input and formatted text

https://mmutube.mmu.ac.uk/id/1_5mg3pkgz

It can be useful to mix text and values on a single line in a print statement. e.g. instead of printing:

```
The value of x is:  
45
```

it is tidier to be able to print:

```
The value of x is 45.
```

There are several ways to do this, the most compact and flexible is by using *formatted text*. For example:

Shell

```
x = 45  
print(f"The value of x is {x}.")
```

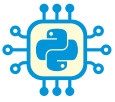
Note that the string of characters given to `print` is prefixed by the letter 'f' before the opening quotation mark indicating that this is formatted text. Anything within the quotes enclosed in curly brackets is not printed verbatim but is interpreted as the value of the variable within the brackets, `x` is this case.

There can be more than one variable included in the output, e.g.:

Shell

```
x = 3  
y = 4  
print(f"The value of y times x = {y} times {x} = {y*x}.")
```

This should output: The value of y times x = 4 times 3 = 12.



2.6 Control of Flow

Each of the scripts so far executes the entire list of instructions one by one and then exits the script. Often, we want to control the flow of the script, e.g. only executing a block of code if a condition is met, or repeating a block of code a number of times, etc.

2.6.1 If

The first example of control of flow that we will look at is the simplest one: the `if` statement. Try out the code below:

```
1 x = int(input("Enter the value of x: "))
2 if x > 10:
3     print("x is bigger than 10")
4 else:
5     print("x is not bigger than 10")
```

[Source Code](#)

Note that there is a link to download the source code directly for this example so you do not need to copy it all into the editor by hand.

The block of code following `if condition:` will only be executed if *condition* is true. Any code following a subsequent, optional `else:` statement will only be executed if *condition* is false.

In this case, if you enter a number for 'x' that is bigger than 10, the first message, "x is bigger than 10" will be displayed; if 'x' is less than or equal to 10, the second message, "x is not bigger than 10" is displayed.

Important note 1: A 'block' of code is signified by the indentation from the left margin. In this example, each print statement forms a single-line block of code. **If you miss out the indentation, this will not work** (the interpreter will show an error).

Important note 2: All Python statements that are followed by an associated block of code like this have a colon (':') at the end of the line. Don't forget to include this.

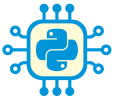
Exercise: Write a script that asks the user to enter two numbers: x and y . The script should then tell the user whether or not $2x$ is less than $3y$

An additional, optional, keyword that can be useful is `elif` - short for 'else if'. An example of its use is:

```
1 x = int(input("Enter the value of x: "))
2 if x > 20:
3     print("x is bigger than 20")
4 elif x > 10:
5     print("x is not bigger than 20 but it is bigger than 10")
6 else:
7     print("x is not bigger than either 10 or 20")
```

[Source Code](#)

There can be more than one `elif` statement following an `if` when you want to test for multiple conditions.



2.6.2 Conditional and Logical Operators and Variables

In the previous exercises, we have used the less-than operator, `<`. This is one of a family of similar operators you can use in `if` statements. The full list is: Often, we need to combine two or more

Operator	Meaning
<code>==</code>	Is equal to
<code>!=</code>	Is not equal to
<code><</code>	Is less than
<code>></code>	Is greater than
<code><=</code>	Is less than or equal to
<code>>=</code>	Is greater than or equal to

conditions using logical operators. The most useful are:

Operator	Meaning
<code>and</code>	<i>condition1</i> and <i>condition2</i> is True only if both conditions are True
<code>or</code>	<i>condition1</i> or <i>condition2</i> is True only if either condition is True
<code>not</code>	not <i>condition</i> is True only if the condition is False

For example, to determine whether an inputted integer, *k*, is in the fifties, you could write:

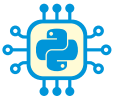
```
1 print("Please input an integer:")
2 k = int(input())
3 if k >= 50 and k < 60:
4     print(f"The number, {k}, is in the fifties.")
```



Video Guide: Conditional and logical operators
https://mmutube.mmu.ac.uk/id/1_h4a5fmfu

Exercise: Write a script that inputs three integers, *x*, *y* and *z*, and displays:

- "x is the largest" if x is larger than y and x is larger than z
- "y is the largest" if y is larger than x and y is larger than z
- "z is the largest" if z is larger than x and z is larger than y
- "Dead heat" if none of the above is true (e.g. x = 5, y = 3, z = 5)



Sometimes, we want to store the result of a comparison in a variable. This is what the *bool* data type is for. Remember, bool (short for Boolean) variables only have two possible values: True or False. Try the following as an example:

```
1 a = 3
2 b = 6
3 c = (b > a)
4 print(f"The value of c is {c}.")
```

(6 is greater than 3 so the value of c should be displayed as 'True')

2.6.3 While

The `while()` statement allows you to repeat a block of code for as long as a *condition* is true. For example:

```
1 i = 0
2 while i < 10:
3     print(i)
4     i = i + 1
```

You should see a list of the numbers from 0 to 9 displayed.

The block of code following the `while` statement consists of two lines this time (both must be indented by the same amount). The first prints the current value of `i`, and the second increases the current value of `i` by one. These lines are repeated, in sequence, as long as `i < 10`. When `i` equals 10, the condition is no longer true and the script ends.

Exercise: Write a script that lists all the powers of 3 (i.e. 1, 3, 9, 27, 81, etc.) less than one million. Use a variable (e.g. 't') initialised to one and a `while()` loop that displays `t` and triples it on each iteration, and repeats while `t` is less than one million.

2.6.4 For

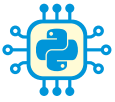
The `for` statement will execute a block of code once for every value of a variable in a *sequence*. A simple example, equivalent to the `while()` example above is:

```
1 for x in (0,1,2,3,4,5,6,7,8,9):
2     print(x)
```

The block of code following the `for` statement will be executed multiple times with the variable `x` taking the value of each item in the list in brackets in sequence.

The numbers in the sequence can be in any order that you wish. Often, however, we want the numbers to go up in series over a range of values and a shorthand can be used:

```
1 for x in range(10):
2     print(x)
```



The `range()` function can be used in three different ways depending how many parameters are given:

Single parameter: `range(10)` equivalent to: (0,1,2,3,4,5,6,7,8,9)

With start parameter: `range(3,10)` equivalent to: (3,4,5,6,7,8,9)

With a step parameter: `range(3,10,2)` equivalent to: (3,5,7,9)

Exercise: Write a script that uses a `for()` loop to display the first 20 square numbers, starting at 1. Use formatted text to display the results in the format:

```
1 × 1 = 1
2 × 2 = 4
3 × 3 = 9
etc.
```

2.7 Comments

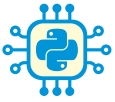
Comments are lines of code that are ignored by the Python interpreter and are included purely to make the code more readable to other people. In Python a comment can be inserted by typing a '#' symbol. Everything after that point on a line will be ignored. E.g.

```
1 # This line is a comment and will not be evaluated.
2
3 v = 4 # Everything after '#' is ignored.
```

You should get into the habit of including useful comments in your code but not to the point that they make the code difficult to read. At the very least, your code should always contain comments at the start saying what the script does, who wrote the code, etc.

If your comment is a long one that spans multiple lines, you will either need to use a '#' character at the start of each line or enclose the whole block of text in triple quotes, e.g.

```
1 """
2 This is a very long comment
3 that runs over several lines,
4 none of which will be evaluated
5 by the interpreter.
6 """
7
8 print("This is the actual script")
```



2.8 Functions



Video Guide: Functions

https://mmutube.mmu.ac.uk/id/1_afycyju2

A *function* is a block of code that does not run until it is *called* from another part of the script. They were introduced earlier in this session using the example built-in functions: `print()`, `abs()`, `max()` and `help()`. As well as using built-in functions, you can also define your own. Functions are defined using the `def` keyword (short for 'define'). For example:

```
1 def test():
2     print("Function test output.")
```

Running this code will not appear to do anything. However, you can now call the function, `test()`, either in the script following the function definition, or directly from the Shell. In the Shell window, type:

Shell

```
test()
```

and the function should be called (displaying the message on line 2). Functions can be written so that one or more parameters, known as *arguments*, are passed in the call. For example:

```
1 def showDouble(x):
2     print(f"{x} doubled is {x * 2}")
3
4 showDouble(3)           # Prints "3 doubled is 6"
5 showDouble(21)          # Prints "21 doubled is 42"
```

Functions can take multiple arguments, separated by commas, and can return a value too. For example:

```
1 def addAndDouble(a, b):
2     x = (a + b) * 2
3     return x
4
5 result = addAndDouble(3, 4)
6 print(result)
```

This example declares a function that adds together two arguments, doubles the sum and returns the result. Calling it with arguments 3 and 4 should return $(3+4) \times 2 = 14$.

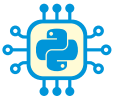
At the moment, if you try using the `help()` function on any of your functions, you won't get much information. You can tell the help function what to report by including a *docstring* comment immediately after the `def` statement, e.g. modify and run the 'addAndDouble' script as follows:

```
1 def addAndDouble(a, b):
2     """Adds together a and b and returns the sum doubled."""
3     x = (a + b) * 2
4     return x
```

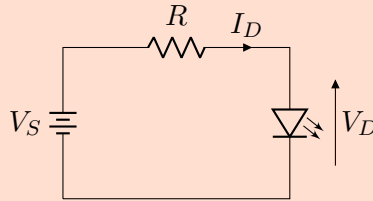
and now, to see your custom help text, type in the shell window:

Shell

```
help(addAndDouble)
```



Task: The circuit below shows a common configuration used to drive an LED (e.g. from a microcontroller output). The resistor, R is required to limit the LED current - omitting it will very likely destroy the LED.



Calculation of the correct resistor depends on the voltage supplied, V_S and the *Forward Voltage* of the LED, V_D , at the required current, I_D - these values can be found on LED datasheets. R is calculated using Ohm's law:

$$R = \frac{V_S - V_D}{I_D}$$

1. Write a function, `calculate(Vs, Vd, I)`, that takes the supply voltage, LED forward voltage and required current as input parameters and returns the resistor value required.
2. Test your function for the following test cases:
 - (a) Red LED, 3.3V supply: `calculate(3.3, 2, 0.02)` [65 Ω]
 - (b) Blue LED, 5V supply: `calculate(5, 3, 0.02)` [100 Ω]
 - (c) Super bright white LED, 6V supply: `calculate(6, 3.3, 0.15)` [18 Ω]
3. Test and submit your function via the link in this week's Moodle section.

Exercise: Mersenne Primes

A Mersenne Prime is a number that is prime and that can be represented as $2^n - 1$ where n is an integer. Find the first seven Mersenne primes by:

1. Write a function, `isFactor(x, d)`, that returns True if the first argument, x , is exactly divisible by d . *Hint: $x \% d == 0$ tests for exact divisibility.*
2. Write a second function, `isPrime(y)`, that returns True if the argument, y , is a prime number. Use a `for` loop to test every integer less than y using a call to your first function, `isFactor()`, to perform the test. If all `isFactor()` calls for the number return False, the number must be prime.
3. Write a script that uses your functions to iterate through the sequence $2^n - 1$, starting at $n = 0$, to find the first seven Mersenne primes. Use another variable to keep count of the number found so far and a `while` loop to stop when seven are found (the seventh should be 524287)
4. (optional) There are more efficient ways of testing for the primeness of a number. Experienced coders may like to research and/or invent one and implement it.